

DcSim Electrical Model Overview – v0.4

This section of the USER_GUIDE replaces the separate "Basic Bus Models" document to avoid duplication and ensure all technical details are centralized.

Purpose

The purpose of DcSim is to simulate how train traffic interacts with the electrical infrastructure in terms of power flows, voltage stability, losses, and energy usage. The model supports realistic DC electrical modeling, multiple trains, time-stepped simulation, and detailed energy flow tracking.

DcSim Electrical Model Overview

Substations

- Represented as ideal EMFs with internal resistance.
- Connected between `groundNodeId` and the specified node.
- Supply power only when voltage at feeding node is below EMF.
- No backfeed allowed (diode behavior).

DC Lines

- Modeled as purely resistive elements.
- Resistance proportional to physical length.
- Power loss computed as I^2R .

Connection Points

- Represent physical points along the track where line characteristics change.
- Examples: change in number of catenaries, change in resistance per km.
- No electrical parameters themselves; characteristics are defined in connected lines.

Trains

- Two-port, power-controlled devices.
- Support both traction and regenerative braking.
- Transition between line regeneration and braking resistor.
- Power request based on spline-interpolated, time-dependent profile.

Braking Logic

- Regeneration to line if voltage < regeneration cutoff (e.g., 850 V).
- Above maximum voltage (e.g., 1000 V), all braking power goes to resistor.
- Between thresholds, power split linearly between line and resistor.

Power Profiles

- Defined per train template, loaded from `.xlsx`.
- Required columns: `time [s]`, `bisPosition [km,m]`, `speed [m/s]`, `primaryMotoringPower [kW]`, `primaryMotorBrakingPower [kW]`.
- Auxiliary power extracted during station dwells.
- If `motoringAndAuxiliariesInSameModel = false`, additional demand added to all generated loads.

Output Variables

- Node voltages per timestep.
- Device currents and powers (substations, trains, lines).
- Train power balance: line power, brake resistor power.
- All outputs exportable to CSV and/or Excel.

Configuration Structure and Examples

The configuration is defined in `application.conf` under the `dcsim` root. Below, each section includes a description and a minimal working example.

1. track

- **stations** (List): Each object represents a station.
 - `name` (String)
 - `abbreviation` (String)
 - `position` (String): "<line_number> km+meters" format, e.g., `1 0+0`.
 - `description` (String, optional)

Example:

```
track {
  stations = [
    { name = "Station A", abbreviation = "SA", position = "1 0+0" },
    { name = "Station B", abbreviation = "SB", position = "1 10+0" }
```

```
]
}
```

2. grid

- **groundNodeId** (Int): Node ID connected to ground.
- **nodes** (List):
 - **id** (Int)
 - **position** (String)
 - **description** (String, optional)
- **lines** (List):
 - **from / to** (Int)
 - **resistance** (Double)
 - **description** (String, optional)
- **substations** (List):
 - **id** (String)
 - **nodeId** (Int)
 - **emf** (Double)
 - **internalResistance** (Double)
 - **description** (String, optional)
 - **Note:** Connected between **groundNodeId** and **nodeId**.
- **connectionPoints** (List):
 - **position** (String)
 - **description** (String, optional)

Example:

```
grid {
  groundNodeId = 0
  nodes = [
    { id = 0, position = "1 0+0" },
    { id = 1, position = "1 10+0" }
  ]
  lines = [
    { from = 0, to = 1, resistance = 0.076 }
  ]
  substations = [
    { id = "SS1", nodeId = 0, emf = 750.0, internalResistance = 0.014 }
  ]
  connectionPoints = [
    { position = "1 5+0", description = "Transition to double catenary" }
  ]
}
```

```
]
}
```

3. traffic

- **timetable**
 - `trains` (List)
 - `id` (String)
 - `templateId` (String)
 - `departure` (String, date/time optional timezzone)
 - `headway` (String)
 - `count` (Int)
 - `signature` (String, optional)
 - `description` (String, optional)
- **templates** (Map)
 - Key: `templateId`
 - Value: object with `stops`

Example:

```
traffic {
  timetable {
    trains = [
      { id = "1023", templateId = "T1", departure = "2025-08-11T08:00:00+02:00",
headway = "00:05:00", count = 1 }
    ]
  }
  templates = {
    T1 = {
      stops = [
        { signature = "SA", departure = "08:00:00" },
        { signature = "SB", arrival = "08:10:00" }
      ]
    }
  }
}
```

4. powerProfiles

- **motoringAndAuxiliariesInSameModel** (Bool)
- **auxiliaryPower** (Double, kW):

- During dwells, extracts given power.
- If `false`, adds extra demand to all loads.
- **templates** (List)
- **Profile files** must have:
 1. `time [s]`
 2. `bisPosition [km,m]`
 3. `speed [m/s]`
 4. `primaryMotoringPower [kW]`
 5. `primaryMotorBrakingPower [kW]`

Example:

```
powerProfiles {
  motoringAndAuxiliariesInSameModel = false
  auxiliaryPower = 50.0
  templates = [
    {
      id = "T1"
      folder = "input/loads/T1"
      legs = [
        { fromStation = "SA", toStation = "SB", file = "SA-SB.xlsx" }
      ]
    }
  ]
}
```

5. simulationControl

- **tickDuration** (Double)
- **simulationStart** (String)
- **simulationEnd** (String)
- `description` (String, optional)

Example:

```
simulationControl {
  tickDuration = 1.0
  simulationStart = "2025-08-11T08:00:00+02:00"
  simulationEnd = "2025-08-11T09:00:00+02:00"
}
```

6. How to Run

Purpose: `DcSimApp` runs the simulation based on your configuration file.

Using sbt:

```
sbt runMain dcsim.DcSimApp
```

Using IntelliJ:

1. Open the project.
2. Navigate to `DcSimApp` .
3. Right-click and choose **Run**.

7. System Requirements

- Java 17+
- Scala 2.13
- sbt
- Excel `.xlsx` power profile files

Usage

- Place `application.conf` at `project/<project_name>/` .
- Follow this guide for all edits.
- See `minimalTest` for a small example.